

Documentación Técnica: TAD E-commerce Magenta

Pablo Saez Morales
Oleksandr Baranets
Nicolas Porra Collado

10 de Mayo de 2026

Repositorio: <https://github.com/pablosaez21/TAD-Ecommerce>
Grupo Magenta: Pablo Saez Morales, Oleksandr Baranets, Nicolas Porra Collado

Índice general

Capítulo 1

Resumen ejecutivo

TAD E-commerce Magenta es una aplicación de comercio electrónico desarrollada con **Laravel 9** que soporta catálogo de productos, carrito en sesión, proceso de checkout con transacciones ACID, gestión de direcciones, panel administrativo y sistema multiidioma (es/en). Esta documentación recoge diseño, decisiones técnicas, estructura, instrucciones de instalación y uso, pruebas y conclusiones.

Capítulo 2

Identidad de marca

2.1. Concepto de marca

La marca **Magenta** se define alrededor de tres ideas: energía, cercanía y modernidad técnica. En el contexto del e-commerce, esto se traduce en una interfaz clara, un proceso de compra rápido y una narrativa visual coherente en todo el sitio.

2.2. Psicología del color

El color principal del proyecto es #FF007F (magenta). La elección se justifica por:

- **Energía y dinamismo:** refuerza una percepción activa, alineada con catálogo deportivo.
- **Diferenciación:** evita paletas neutras típicas de tiendas genéricas.
- **Reconocimiento de marca:** facilita consistencia visual en navegación, botones y estados.

Se complementa con #C80064 para contraste y jerarquía visual en enlaces y títulos.

2.3. Logo

Esta versión representa el logotipo verbal utilizado en la documentación y puede sustituirse por un recurso corporativo cuando se incorpore al repositorio.



Figura 2.1: Logotipo de Double Helix.

Capítulo 3

Objetivos y alcance

3.1. Objetivos

- Implementar un e-commerce funcional con autenticación segura (Fortify).
- Asegurar coherencia de datos en el checkout mediante transacciones y bloqueo de stock.
- Proveer un panel administrativo para gestión de catálogo y pedidos.
- Mantener buenas prácticas de seguridad y rendimiento.

3.2. Alcance

El alcance cubre las funcionalidades implementadas en el repositorio actual: modelos, migraciones, controladores, vistas Blade, rutas, internacionalización y envíos de correo (Mailables). No incluye integraciones de terceros en producción (pasarelas de pago reales) aunque la arquitectura permite su incorporación.

Capítulo 4

Casos de uso

4.1. Actores

- **Visitante:** navega catálogo y detalle sin autenticación.
- **Usuario autenticado:** compra, guarda favoritos, gestiona perfil y pedidos.
- **Administrador:** gestiona catálogo y supervisa métricas en dashboard.

4.2. Casos de uso principales

Caso	Descripción ajustada al código actual
UC-01 Ver catálogo	El usuario consulta <code>/products</code> con paginación y filtros por categoría.
UC-02 Gestionar carrito	En <code>CartController</code> , el sistema añade o elimina productos en sesión, limitando cantidad al stock.
UC-03 Checkout	En <code>CartController::checkout</code> , se valida address y items, se bloquea stock con <code>lockForUpdate()</code> , se crea orden, líneas y pago en una transacción.
UC-04 Gestión de perfil	En <code>ProfileController</code> , el usuario administra direcciones, idioma y consulta pedidos.
UC-05 Favoritos	En <code>FavoriteController</code> , el usuario alterna favoritos sobre productos.
UC-06 Administración	Rutas protegidas por <code>auth + admin</code> para dashboard y CRUD de productos/categorías.

4.3. Caso de uso crítico: Comprar productos

1. Usuario autenticado abre checkout.
2. Sistema valida dirección del usuario y contenido de `items`.
3. Por cada item, bloquea fila del producto y verifica stock.
4. Crea `orders`, `order_lines` y `payments` dentro de `DB::transaction`.
5. Vacía carrito de sesión y envía `OrderConfirmationMail`.

Capítulo 5

Arquitectura y diseño

5.1. Visión general

La aplicación sigue el patrón MVC de Laravel: modelos Eloquent para la lógica de datos, controladores para orquestación, vistas Blade para presentación. Los assets se gestionan con Vite y npm.

5.2. Modelos principales

Resumen de modelos y relaciones (implementadas en `app/Models`):

- **User**: relaciones `hasMany(addresses, orders)`, `belongsToMany(products)` via `favorites`.
- **Product**: relaciones `belongsToMany(categories)`, `hasMany(orderLines)`, campos: `name`, `description`, `price`, `stock`, `active`.
- **Category**: `belongsToMany(products)`.
- **Order**: `belongsTo(user, address)`, `hasMany(orderLines)`, `hasOne(payment)`.
- **OrderLine**: `belongsTo(order, product)`, guarda `quantity`, `unit_price`, `subtotal`. **Address** : `belongsTo(user)` *con campos de dirección y is default*. **Payment** : `belongsTo(order)` *registra el método y es*

5.2.1. Detalles y extractos de código

Los siguientes extractos provienen de los ficheros reales en `app/Models` y ayudan a entender decisiones concretas.

Producto (`app/Models/Product.php`)

El modelo define los campos rellenables y las relaciones principales:

```
1 protected $fillable = ['name', 'description', 'price', 'image', 'stock', 'active'];
2
3 public function categories(): BelongsToMany { return $this->belongsToMany(
    ↪ Category::class); }
```

```

4 public function orderLines(): HasMany { return $this->hasMany(OrderLine::class
    ↪ ); }
5 public function favoredByUsers(): BelongsToMany { return $this->belongsToMany(
    ↪ User::class, 'favorites'); }

```

Notas:

- El campo `price` se castea a decimal con dos decimales para evitar errores de formato.
- La relación `favoredByUsers` usa la tabla pivote `favorites`.

Orden (app/Models/Order.php)

El modelo de orden relaciona líneas, usuario, dirección y pago:

```

1 protected $fillable = ['user_id', 'address_id', 'status', 'total'];
2
3 public function lines(): HasMany { return $this->hasMany(OrderLine::class); }
4 public function payment(): HasOne { return $this->hasOne(Payment::class); }

```

Observaciones:

- El total se almacena con precisión decimal (`decimal:2`).
- Se mantiene copia de precio en `order_lines` para auditoría histórica.

5.3. Esquema de base de datos

5.3.1. Diagrama UML de base de datos

El diagrama refleja las relaciones implementadas en las migraciones y modelos: **User-Order** (1:N), **Order-OrderLine** (1:N), **Order-Payment** (1:1) y relaciones N:M mediante tablas pivote (p.ej. `favorites`, `category_product`).

Se aplica normalización hasta 3NF; las tablas pivot eliminan redundancia en relaciones N:M (ej. `favorites`, `category_product`, `discount_product`). Las decisiones principales:

- Precios se copian en `order_lines` para auditoría histórica.
- Las claves foráneas usan `onDelete('cascade')` cuando procede.
- Índices en columnas de búsqueda (`created_at`, `status`).

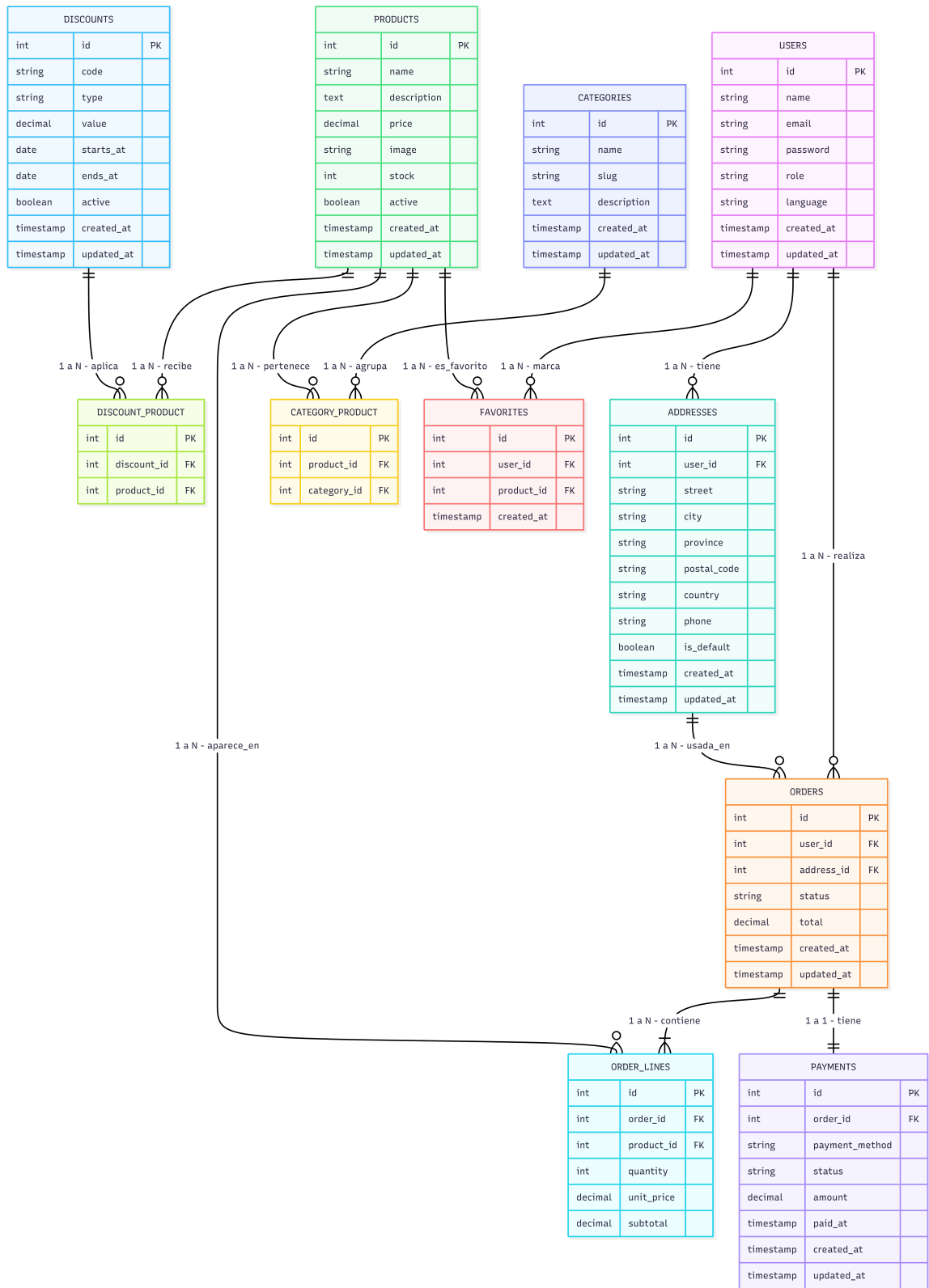


Figura 5.1: Modelo UML de base de datos (fuente: docs/uml.png).

Capítulo 6

Rutas y controladores

6.1. Rutas principales

Rutas definidas en `routes/web.php` (resumen):

- PÚBLICAS: GET `/`, GET `/products`, GET `/products/product`, GET `/categories/slug`.
- AUTENTICADAS: GET/POST `/cart`, POST `/cart/checkout`, perfil y direcciones, favoritos.
- ADMIN: rutas protegidas con middleware `admin` para `/admin` y CRUD de productos y categorías.

6.2. Controladores clave

- **ProductController**: list, show, create, store, edit, update, destroy (admin + público).
- **CartController**: gestionar carrito en sesión, validar stock, checkout (transacción).
- **ProfileController**: gestionar direcciones, idioma, ver pedidos.
- **FavoriteController**: toggle favoritos.
- **DashboardController**: estadísticas administrativas.

6.2.1. CartController: métodos clave

Se muestran extractos reales de `app/Http/Controllers/CartController.php` y una explicación del comportamiento:

Añadir al carrito (add)

El método valida disponibilidad del producto y mantiene el carrito en sesión. Fragmento:

```

1 if (! $product->active || $product->stock <= 0) {
2     return back()->with('error', 'Este producto no est disponible.');
```

Notas:

- El carrito se guarda como array asociativo en la sesión: clave = id del producto.
- La cantidad se clampa por el stock disponible para evitar solicitudes inválidas.

Checkout (checkout)

El checkout es el núcleo del flujo de negocio: usa validación, transacción y bloqueo pessimistico para stock.

```

1 DB::transaction(function () use ($validated, $user, $address) {
2     $order = Order::create([...]);
3     foreach ($validated['items'] as $item) {
4         $product = Product::whereKey($item['product_id'])
5             ->where('active', true)
6             ->lockForUpdate()
7             ->firstOrFail();
8
9         if ($product->stock < $item['quantity']) {
10             abort(422, "Stock insuficiente para {$product->name}");
11         }
12
13         $order->lines()->create([ ... ]);
14         $product->decrement('stock', $item['quantity']);
15     }
16     $order->payment()->create([...]);
17     return $order->fresh(['lines.product', 'payment', 'address']);
18 });
```

Explicación:

- Se usa `lockForUpdate()` para bloquear la fila del producto durante la transacción.
- Si falta stock, se aborta con código 422, provocando rollback.
- La creación del pago se realiza dentro de la misma transacción para mantener consistencia.

Capítulo 7

Proceso de compra (checkout)

7.1. Flujo normal

1. Usuario autenticado inicia checkout desde carrito en sesión.
2. Sistema valida stock de cada producto con bloqueo (`lockForUpdate`).
3. Se crea la orden, `order_lines` y registro de payment dentro de una `DB::transaction`.
4. Se decrementa stock y se actualiza total.
5. Se envía correo de confirmación (Mailable) y se limpia el carrito.

7.2. Manejo de errores y rollback

Cualquier fallo en la transacción provoca rollback. Casos alternativos: carrito vacío, stock insuficiente, fallo en envío de correo (reintentos asincrónicos / logging).

Capítulo 8

Decisiones técnicas y de diseño

8.1. Autenticación

Se eligió **Laravel Fortify** para autenticación y gestión de credenciales; permite 2FA y acciones personalizadas (`CreateNewUser`, `UpdateUserPassword`, etc.).

8.2. Carrito en sesión

Ventaja: simple, rápida y suficiente para la EPD. Alternativa (persistente en BD) considerada para futuras versiones.

8.3. Transacciones y concurrencia

Uso de transacciones ACID y `lockForUpdate` para evitar sobreventa de stock en escenarios concurrentes.

8.4. Internacionalización

Se utilizan archivos PHP en `resources/lang/es` y `resources/lang/en`. La preferencia de idioma se guarda en el campo `language` del usuario.

Capítulo 9

Estructura de ficheros relevante

- `app/Models/`: modelos Eloquent.
- `app/Http/Controllers/`: controladores.
- `resources/views/`: plantillas Blade (productos, carrito, checkout, perfil, admin).
- `routes/web.php`: rutas web.
- `database/migrations/`: migraciones y seeders.
- `app/Mail/`: clases Mailable (OrderConfirmationMail).

Capítulo 10

Instalación y puesta en marcha

10.1. Requisitos

- PHP 8.1+
- Composer
- Node.js 16+ y npm
- MySQL o PostgreSQL

10.2. Pasos rápidos

```
# clona el repo (local ya presente)
cd /home/miller/Desktop/tad/TAD-Ecommerce
composer install
cp .env.example .env
php artisan key:generate
# configurar .env con credenciales DB y MAIL
php artisan migrate --seed
npm install
npm run dev
php artisan serve
```

over: <http://localhost:8000>

Capítulo 11

Seguridad

La seguridad del proyecto no se limita a una sola capa, sino que se distribuye en autenticación, autorización, validación, control de concurrencia y protección de datos de negocio.

11.1. Autenticación y contraseñas

El sistema usa Laravel Fortify para centralizar registro, inicio de sesión, cambio de contraseña y, si se activa, autenticación en dos pasos. Las contraseñas nunca se almacenan en texto plano, sino mediante hashing seguro gestionado por Laravel. Esto reduce el impacto de una posible fuga de base de datos y permite reutilizar mecanismos nativos del framework.

11.2. Autorización por rol

Las rutas administrativas están protegidas por middleware y por la comprobación del rol del usuario. En la práctica, esto evita que un usuario normal pueda acceder al panel de administración, editar productos o visualizar estadísticas internas. El control está alineado con el campo `role` del modelo `User`.

11.3. Protección frente a ataques web

- **CSRF:** los formularios Blade incluyen token `@csrf`, evitando peticiones forjadas desde sitios externos.
- **Validación server-side:** las peticiones se validan antes de persistir datos, lo que impide enviar valores vacíos, tipos incorrectos o IDs inexistentes.
- **Escape en vistas:** Blade escapa la salida por defecto, reduciendo el riesgo de XSS cuando se muestran nombres, descripciones o mensajes.

11.4. Seguridad en el checkout

El punto más sensible del sistema es la compra. Por eso, el `CartController` valida dirección, métodos de pago y contenido del carrito antes de abrir la transacción. Además,

cada producto se bloquea con `lockForUpdate()` para evitar que dos compras simultáneas resten el mismo stock. Si el stock no alcanza, la operación falla antes de completar la orden.

11.5. Integridad y trazabilidad

- El pedido, sus líneas y su pago se crean dentro de la misma transacción, evitando estados inconsistentes.
- El stock se decrementa solo tras confirmar que el producto está activo y disponible.
- El correo de confirmación se envía después de confirmar la compra, evitando notificaciones falsas.

11.6. Medidas adicionales recomendables

Aunque el proyecto ya es seguro para un entorno académico, en un escenario de producción se recomienda añadir:

- Rate limiting en login y checkout.
- Logs de auditoría para acciones administrativas.
- Política de contraseñas más estricta si el requisito del curso lo exige.
- Validación de ficheros subidos para imágenes de productos.

Capítulo 12

Decisiones de mejoras futuras

El proyecto está preparado para crecer sin reescribir su base principal. Las siguientes mejoras son las más naturales y útiles a partir del código existente:

12.1. Pagos y confirmación real

La versión actual guarda el método de pago y marca el pago como `paid` dentro de la propia aplicación. El siguiente paso lógico sería integrar una pasarela real como Stripe o PayPal, de forma que el estado del pago dependa de una respuesta externa verificada mediante webhook. Esto permitiría separar mejor la intención de compra del pago efectivo.

12.2. Carrito persistente

Actualmente el carrito vive en sesión, lo cual es apropiado para un proyecto simple y rápido. Como mejora, podría guardarse en base de datos para que el usuario recupere su carrito desde otro dispositivo o después de cerrar sesión. Esta evolución sería especialmente útil en tiendas con usuarios recurrentes.

12.3. Gestión avanzada de pedidos

Se podría ampliar el ciclo de vida del pedido con más estados y trazabilidad: pendiente de pago, pagado, preparando, enviado, entregado y cancelado. Esto encaja con la tabla `orders` y permitiría al panel administrativo gestionar mejor el proceso logístico.

12.4. Búsqueda, filtros y recomendación

El catálogo ya parte de una estructura limpia de productos y categorías. Sobre esa base se puede añadir:

- Buscador por nombre o descripción.
- Filtros combinados por categoría, precio y estado.
- Productos relacionados o recomendados por historial de compra.

12.5. Calidad, pruebas y despliegue

Para acercar el proyecto a un entorno profesional, conviene reforzar la calidad técnica con:

- Tests de integración para checkout, favoritos y administración.
- CI/CD automático con GitHub Actions.
- Seeds más ricos para demostrar escenarios de negocio reales.
- Revisión de accesibilidad y rendimiento en móviles.

12.6. Internacionalización ampliada

El soporte actual en español e inglés puede crecer con más traducciones en componentes dinámicos, mensajes de error y correos. Esto haría que la experiencia sea realmente consistente en ambos idiomas, no solo en la interfaz principal.

Capítulo 13

Apéndice: fragmentos de código relevantes

13.1. Ejemplo: Transacción de checkout (extracto)

```
1 DB::transaction(function() use ($cart, $user, $validated) {  
2     foreach ($cart as $item) {  
3         $product = Product::where('id', $item['id'])  
4             ->lockForUpdate()->first();  
5         if ($product->stock < $item['quantity']) {  
6             throw new Exception('Stock insuficiente');  
7         }  
8     }  
9     $order = Order::create([...]);  
10    // crear order lines, decrementar stock, crear payment  
11 });
```

Capítulo 14

Conclusión

Esta documentación proporciona una base completa y profesional del proyecto TAD E-commerce Magenta. Si quieres, puedo:

- Añadir diagramas simplificados (tablas) para arquitectura y casos de uso.
- Generar un PDF (compilar) desde aquí — avisando que la compilación puede fallar si faltan paquetes del sistema.
- Eliminar físicamente ficheros de imagen no deseados del repositorio.

Documento preparado por Grupo Magenta — Mayo 2026